

PROJECT REPORT

Deep Vision

YOLOX and YOLOv8/9/10/11 Model Exploration

Plastic Shampoo Bottle Defect Detection

Content Submitted and Authored by

Dhruvin Gopani
d.gopani@oth-aw.de

Khushalkumar Variya
k.variya@oth-aw.de

Mentored by

Prof. Dr. Tatyana Ivanovska

Ostbayerische Technische Hochschule Amberg-Weiden

Department of Electrical Engineering, Media and Computer Science

June 25, 2026

Abstract

Identification of objects is a critical problem in industry applications since the technique is capable of identifying and classifying defects from product images automatically. In this study, five different object detection networks using YOLO architecture have been investigated for plastic shampoo bottles. These networks include YOLOv8n, YOLOv9-C, YOLOv10n, YOLO11n, and YOLOX-Nano. It was intended not just to train the models but also to investigate their performance in terms of accuracy, size, cost of computation.

The dataset contains 10 defect classes: adhesion_marg, adhesion_unacc, dent_marg, dent_unacc, pob_marg, pob_unacc, scuff_marg, scuff_unacc, tear_marg and tear_unacc. Training, validation, and testing of each model occurred on the same detection task. Evaluation is based on the following primary metrics: precision, recall, F1 score, and mAP50. There is also a discussion of other characteristics of the models including layers, parameters, GFLOPS, optimizer, activation function, floating point precision, and fine-tuning properties.

The results indicate that YOLO11n performed the best with the highest F1-Score and mAP50. YOLOv8n performed well on a good balance of accuracy and training time. YOLOv9-C performed well with high accuracy but needed more computational effort. YOLOv10n performed very fast with relatively good accuracy. Finally, YOLOX-Nano had the least number of parameters but performed poorly on detection accuracy compared to the latest YOLO versions.

Contents

Abstract **1**

1 Motivation and Introduction **3**

1.1 Motivation 3

1.2 Introduction 3

2 Related Work **4**

2.1 YOLO Object Detection 4

2.2 Transfer Learning and Fine-Tuning 4

2.3 Loss Functions and Optimization 4

3 Dataset Description and Preprocessing **5**

3.1 Dataset Overview 5

4 Methodology **6**

4.1 General YOLO Architecture 6

4.2 Transfer Learning and Weight Tuning 6

4.3 Parameter Change in the Detection Head 7

4.4 Model Attribute Summary 7

4.5 Loss Functions 8

4.6 Optimizers and Activation Functions 8
4.7 Individual Model Methodology 9
4.7.1 YOLOv8n 9
4.7.2 YOLOv9-C 9
4.7.3 YOLOv10n 9
4.7.4 YOLO11n 10
4.7.5 YOLOX-Nano 10
5 Experimental Setup **11**
5.1 Hardware, Software and Training Configuration 11
5.2 Evaluation Metrics 11
5.3 Training and Validation Plots 11
5.3.1 YOLOv8n Plots 12
5.3.2 YOLOv9-C Plots 12
5.3.3 YOLOv10n Plots 13
5.3.4 YOLO11n Plots 14
5.3.5 YOLOX-Nano Plots 14
6 Results and Evaluation **16**
6.1 Overall Test Results 16
6.2 Best Model for Each Metric 16
6.3 Detection Performance and Efficiency Ranking 16
6.4 Final Summary Graphs 17
7 Discussion, Conclusion and Future Work **19**
7.1 Discussion 19
7.2 Conclusion 19
7.3 Future Work 19
Appendix: Sample Detection Outputs **20**
YOLO11n Detection Output 20
YOLOv10n Detection Output 21
YOLOv8n Detection Output 22

Chapter 1
Motivation and Introduction

1.1 Motivation

The inspection process of industrial quality assurance involves detection of surface flaws in an effective way. In manual inspection, the accuracy of the process is dependent on the attentiveness and the experience level of the person. Deep learning based object detection technology helps overcome such challenges of inconsistency in inspection through automated defect detection and classification.

The reason behind choosing this project is the use of various YOLO models for the plastic shampoo bottle defect detection task. Different models have been chosen to represent the various stages of YOLO development. YOLOv8n has been chosen as a modern baseline model. YOLOv9-C is an advanced gradient information-based model while YOLOv10n concentrates on efficient detection. YOLO11n is a lightweight Ultralytics model.

The main question of this project is:

> Which YOLO model gives the best balance between detection accuracy, model size, computational cost and training time for plastic shampoo bottle defect detection?

The question above is very significant since the optimal model is not necessarily the largest one. While a very big model might perform well in terms of accuracy, it does not have to be the right fit for implementation. Consequently, the following report seeks to compare performance and efficiency.

1.2 Introduction

Object detection is made up of two things: localization and classification. Localization is basically determining the location of the object through the use of a bounding box. Classification is determining the class of the object. For this case, the objects are the defects in the plastic shampoo bottles.

YOLO algorithms are single-stage object detection algorithms. Rather than performing region proposal followed by classification, YOLO algorithms detect the bounding box as well as the corresponding class in a single forward pass. This makes them efficient and ideal for use in real-time applications. YOLO was first proposed by Redmon et al. [1]. There have been several enhancements to YOLO since then.

The current work involves the comparison between the YOLOv8n, YOLOv9-C, YOLOv10n, YOLO11n, and YOLOX-Nano models. These models are compared using the same dataset and the same target classes. This allows for better understanding of the results as differences in performance are mostly due to the model architecture and training procedure.

Chapter 2

Related Work

2.1 YOLO Object Detection

YOLO made the innovation of formulating object detection as a regression problem. It generates bounding box coordinates and the probability of classes in a single pass by using the input image itself [1]. It is faster than the two-stage approach since it bypasses the region proposal step.

Advancements in YOLO frameworks have enhanced this approach through the introduction of improved backbones, better feature fusion, anchor free heads, advanced assignment methods, and effective loss functions. The YOLOv8 framework uses an anchor free split head and offers a good trade-off between accuracy and speed [2]. The YOLOv9 framework adopts the Programmable Gradient Information (PGI) and GELAN methods to improve information flow in training [3]. YOLOv10 framework is geared towards end-to-end object detection with minimal use of NMS [4]. The YOLO11 family of frameworks has been introduced by Ultralytics as newer frameworks for various vision tasks [5].

2.2 Transfer Learning and Fine-Tuning

Transfer learning refers to using a model that has been trained using some useful features on a huge dataset and fine-tuning it according to the new dataset. Yosinski et al. [7] found out that the features extracted using deep neural networks have the capability of transferring between different tasks. This is especially important in object detection since the earlier layers capture generic visual features such as edges, corners, and textures.

For this project, pretrained weights have been utilized wherever available. Pretrained weights will allow the model to have a head start compared to random initialization. The head for final detection is modified for the 10 classes of defects present in the new dataset.

2.3 Loss Functions and Optimization

Detection models generally rely on multiple loss functions. Box loss helps in localization while classification loss helps in classifying predictions, and Distribution Focal Loss (DFL) is applied to newer versions of YOLO models for better box regression. Distribution Focal Loss is a concept where the distribution of bounding box distances is learned rather than just predicting a fixed value [9]. Losses that depend on IoU include DIoU and CIoU [10].

The gradients obtained by backpropagation are used to update the model parameters during training. AdamW is an optimizer that is widely used in deep learning as it uses adaptive learning rate along with decoupled weight decay [8]. The implementation of YOLO sometimes employs SGD with momentum due to its simplicity and stability.

Chapter 3

Dataset Description and Preprocessing

3.1 Dataset Overview

The dataset that will be used in this project is a plastic shampoo bottle defect detection dataset. This dataset is in YOLO format with training, validation and test folders. This dataset configuration has 10 classes.

Table 3.1: Plastic shampoo bottle defect classes.

Class ID	Class Name
0	adhesion_marg
1	adhesion_unacc
2	dent_marg
3	dent_unacc
4	pob_marg
5	pob_unacc
6	scuff_marg
7	scuff_unacc
8	tear_marg
9	tear_unacc

The class names show both the defect type and the severity. For example, `dent_marg` means a marginal dent defect, while `dent_unacc` means an unacceptable dent defect. This makes the task challenging because the model must learn both defect appearance and severity difference.

The dataset was organized in such a way to make it ready for training an object detection network. The images and labels were split into three sets: training set, validation set and test set. The dataset organization is identical for all models, so that the comparison would be unbiased and would reflect the differences in the architecture, training process and capacity of the models only.

Chapter 4

Methodology

4.1 General YOLO Architecture

All the YOLO models used in this project share a common architecture consisting of three main parts: backbone, neck, and detection head. The backbone layer extracts features from images, while the neck layer combines these features across multiple scales.

For an input image of size 640×640, the common detection feature maps are:

Table 4.1: Common feature-map dimensions for YOLO detection.

Feature Level	Stride	Feature Size	Main Purpose
P3	8	80×80	Small defect detection
P4	16	40×40	Medium defect detection
P5	32	20×20	Large defect detection

Small defects are harder to detect because they occupy fewer pixels. Therefore, the 80×80 feature map is important for this dataset.

4.2 Transfer Learning and Weight Tuning

Pretrained weights for these models were used whenever possible. Pretrained weights have visual knowledge already built into them due to large scale training datasets. It is not possible to use all of the weights directly in this model. This is because of the number of classes. While the pre-trained detector will be trained on 80 COCO classes, here there are only 10 classes.

Table 4.2: Which parts are transferred and which parts are newly tuned.

Model Part	Transferred from Pretraining	Tuned on Bottle Dataset
Backbone	Most convolutional filters are transferred.	Fine-tuned to learn bottle material, reflections and defect texture.
Neck	Feature fusion weights are mostly transferred.	Fine-tuned to combine small and large defect features.
Box branch	Many regression weights can be reused.	Tuned to localize defect boxes accurately.
Class branch	Final class-specific weights often do not match.	Trained for the 10 bottle defect classes.
Objectness branch	Used mainly in YOLOX.	Tuned to separate defect regions from background.
DFL branch	Used in newer YOLO models.	Tuned for more accurate box boundary prediction.

The most dataset-dependent tuning will be done at the detection head. That is due to

the reason that the detection head generates the output from the extracted features from the images. The number of classes and shapes of objects present in the dataset affect the outputs.

Learning from scratch implies that the initial weights will be chosen randomly. In such case, a greater amount of training data and more training time are required. It will be better to use fine-tuning because in this case, the pre-trained network already knows how to recognize images in general.

4.3 Parameter Change in the Detection Head

For a class prediction layer with input channels C and number of classes N_c, the number of parameters in a 1×1 convolution is:

$$P_{\text{cls}} = C \cdot N_c + N_c \tag{4.1}$$

The value of $C \cdot N_c$ denotes the weights, while the value of N_c denotes the bias. When there is a pretrained model that has been trained using COCO data set with $N_c = 80$, but the new data set has $N_c = 10$, the size of the last class prediction layer becomes relatively small.

4.4 Model Attribute Summary

Table 4.3: Important model attributes used in this project.

Model	**Optimizer**	**Learning Rate**	**Main Training Idea**
YOLOv8n	AdamW / auto	≈ 0.0014	Anchor-free fine-tuning
YOLOv9-C	SGD / momentum	0.01 initial	Improved gradient flow using PGI
YOLOv10n	AdamW / auto	≈ 0.0014	Efficient end-to-end detection
YOLO11n	AdamW / auto	≈ 0.0014	Balance between accuracy and efficiency
YOLOX-Nano	SGD / momentum	≈ 0.0025	Decoupled head for lightweight detection

The following Table 4.3 presents the major model training parameters. The optimizer for YOLOv8n, YOLOv10n, and YOLO11n was either AdamW or auto-selected since the training was conducted in the Ultralytics setting. The optimizer for YOLOv9-C and YOLOX-Nano was the stochastic gradient descent with momentum.

All models utilized backpropagation for updating the network weights. SiLU was the principal activation function in the YOLO models since it allows smooth gradient descent and aids in learning the small texture of defects. Mixed precision was used in the Ultralytics models where applicable; YOLOv9-C was trained using FP32, while YOLOX-Nano was trained using mixed precision. The actual learning rate may depend on framework and other parameters.

4.5 Loss Functions

Object detection requires several learning objectives. The model must localize the object, classify it and estimate confidence. For the Ultralytics YOLO models, the total loss can be written as:

$$\mathcal{L}_{total} = \lambda_{box}\mathcal{L}_{box} + \lambda_{cls}\mathcal{L}_{cls} + \lambda_{dfl}\mathcal{L}_{dfl} \tag{4.2}$$

where $\mathcal{L}_{box}$ is the box localization loss, $\mathcal{L}_{cls}$ is the classification loss, $\mathcal{L}_{dfl}$ is the Distribution Focal Loss, and λ values control the importance of each loss. YOLOX uses a slightly different loss:

$$\mathcal{L}_{YOLOX} = \lambda_{iou}\mathcal{L}_{iou} + \lambda_{obj}\mathcal{L}_{obj} + \lambda_{cls}\mathcal{L}_{cls} \tag{4.3}$$

Box loss checks how well the predicted box matches the ground truth box. The basic measure is Intersection over Union:

$$IoU = \frac{|B_{pred} \cap B_{gt}|}{|B_{pred} \cup B_{gt}|} \tag{4.4}$$

A simple IoU loss is:

$$\mathcal{L}_{IoU} = 1 - IoU \tag{4.5}$$

Classification loss teaches the model to predict the correct class:

$$\mathcal{L}_{cls} = -\sum_{i=1}^{N_c} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \tag{4.6}$$

Distribution Focal Loss improves bounding box regression. If the true target value y lies between two bins l and r, then:

$$\mathcal{L}_{dfl} = -[(r - y) \log(P_l) + (y - l) \log(P_r)] \tag{4.7}$$

YOLOX uses an objectness branch. The objectness loss is:

$$\mathcal{L}_{obj} = -[o \log(\hat{o}) + (1 - o) \log(1 - \hat{o})] \tag{4.8}$$

4.6 Optimizers and Activation Functions

This part describes the choice of optimizers and activation functions used and how they helped in this project.

AdamW optimizer was chosen for the Ultralytics family of models, including YOLOv8n, YOLOv10n, and YOLO11n. The AdamW is good since it dynamically changes the learning rate and adds weight decay to the training process. It is helpful during fine-tuning when some layers have already learned some important features, but the new detection head should learn 10 bottles' defects.

SGD with momentum was chosen for the YOLOv9-C and YOLOX-Nano architectures. The SGD algorithm is easy to implement and is quite stable in object detection training.

Momentum makes the optimizer continue in the beneficial direction, making the gradient noise less impactful. It is valuable to apply in case of long training sessions of detection models.

SiLU was chosen as the primary activation function for the YOLO models. It is a smooth function that enables better gradient flow than any kind of hard threshold function. That makes it easier to train the network on learning defect textures and bottle surface pattern.

The Sigmoid function was applied in the output branches that require probability-like results. It was convenient to use in confidence score, objectness score, and class probability output cases.

Softmax function was utilized when the model had to spread probabilities across many options. In the new YOLO architectures, Softmax function is important when using Distribution Focal Loss in the bounding box regression task. This will enable the model to learn a probability distribution that can give an accurate prediction of the boundary boxes.

In summary, the above optimization functions and activation functions enabled the models to train stably and apply the learned knowledge in the plastic shampoo bottles defects dataset.

4.7 Individual Model Methodology

4.7.1 YOLOv8n

The first baseline model selected is YOLOv8n. This model is lightweight and consists of 130 layers, 3.01M parameters, and 8.2 GFLOPs. It employs anchor-free detection and DFL-based box regression. This model upgrades previous models like YOLO due to its use of the anchor-free split head, which eliminates the need for manually crafted anchor boxes and makes the training easy. YOLOv8n was selected since it is lightweight, fast, and reliable. After the loading of pretrained weights, the backbone and neck modules exploited general visual features while the classification head was modified for the 10 defects.

4.7.2 YOLOv9-C

The other variant that we have used is called YOLOv9-C and has 962 layers with 51.02M parameters and 239.0 GFLOPS. YOLOv9-C uses Programmable Gradient Information (PGI) and GELAN. Programmable Gradient Information helps improve the quality of gradient information during the training phase. That is beneficial for deep models, where useful gradient information might get diluted after passing through multiple layers. YOLOv9-C was selected due to its ability to have high learning capacity; however, it is also very computationally heavy. The detection head was trained for the 10 bottle defect classes, while the bigger backbone and neck were fine-tuned.

4.7.3 YOLOv10n

YOLOv10n is specifically developed for efficient end-to-end object detection. This network consists of 224 layers, 2.71M parameters, and 8.4 GFLOPs. YOLOv10 was developed to address the dependence on NMS and ensure efficient inference. Regular YOLO networks can generate duplicate bounding boxes, and these are removed using NMS; however, this increases the post-processing time. YOLOv10n was chosen due to its inference efficiency. The pretrained feature extractor was used during the fine-tuning process while adapting the

detection head to 10 defect classes.YOLOv10n is specifically developed for efficient end-to-end object detection.

4.7.4 YOLO11n

YOLO11n is a small state-of-the-art Ultralytics neural network that has 182 layers, consists of 2.59M parameters, and consumes 6.5 GFLOPs. It provides better trade-off between precision and efficiency. YOLO11n was added to verify if a new lightweight neural network is capable of performing better than old or large networks. In this project, it provided the best F1-score and mAP50. The most significant parameter tuning was conducted for the detection head, while backbone and neck were fine-tuned.

4.7.5 YOLOX-Nano

YOLOX-Nano is an anchorless lightweight detector that features decoupled heads with SimOTA label assignments. In this study, YOLOX-Nano had 2.24M parameters and 6.94 GFLOPs. Decoupled heads make a distinction between classification and regression, which is important since these are two different processes. YOLOX-Nano was selected due to its lightness and efficiency. The classification part of the head learned the defect categories, whereas the regression part learned the location of defects. Due to its size, the model capacity is low, which can explain low precision and recall in comparison to other YOLO detectors.

Chapter 5

Experimental Setup

5.1 Hardware, Software and Training Configuration

The computations were conducted on an NVIDIA GeForce RTX 2080 SUPER GPU that supports CUDA. The main training process was implemented with the use of Python, PyTorch and Ultralytics in case of YOLOv8n, YOLOv10n and YOLO11n. YOLOv9-C was trained via the YOLOv9 library, while YOLOX-Nano was trained in the YOLOX environment.

The input images had dimensions 640×640. The dataset was divided into train, validation, and test datasets. The test set was utilized during the main evaluation phase.

Table 5.1: Training configuration summary.

Model	Epochs	Image Size	Main Framework
YOLOv8n	100	640	Ultralytics
YOLOv9-C	50	640	YOLOv9 repository
YOLOv10n	100	640	Ultralytics
YOLO11n	100	640	Ultralytics
YOLOX-Nano	100	640	YOLOX

5.2 Evaluation Metrics

Precision measures how many predicted detections are correct:

$$Precision = \frac{TP}{TP + FP} \tag{5.1}$$

Recall measures how many ground-truth objects were detected:

$$Recall = \frac{TP}{TP + FN} \tag{5.2}$$

F1-score combines precision and recall:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \tag{5.3}$$

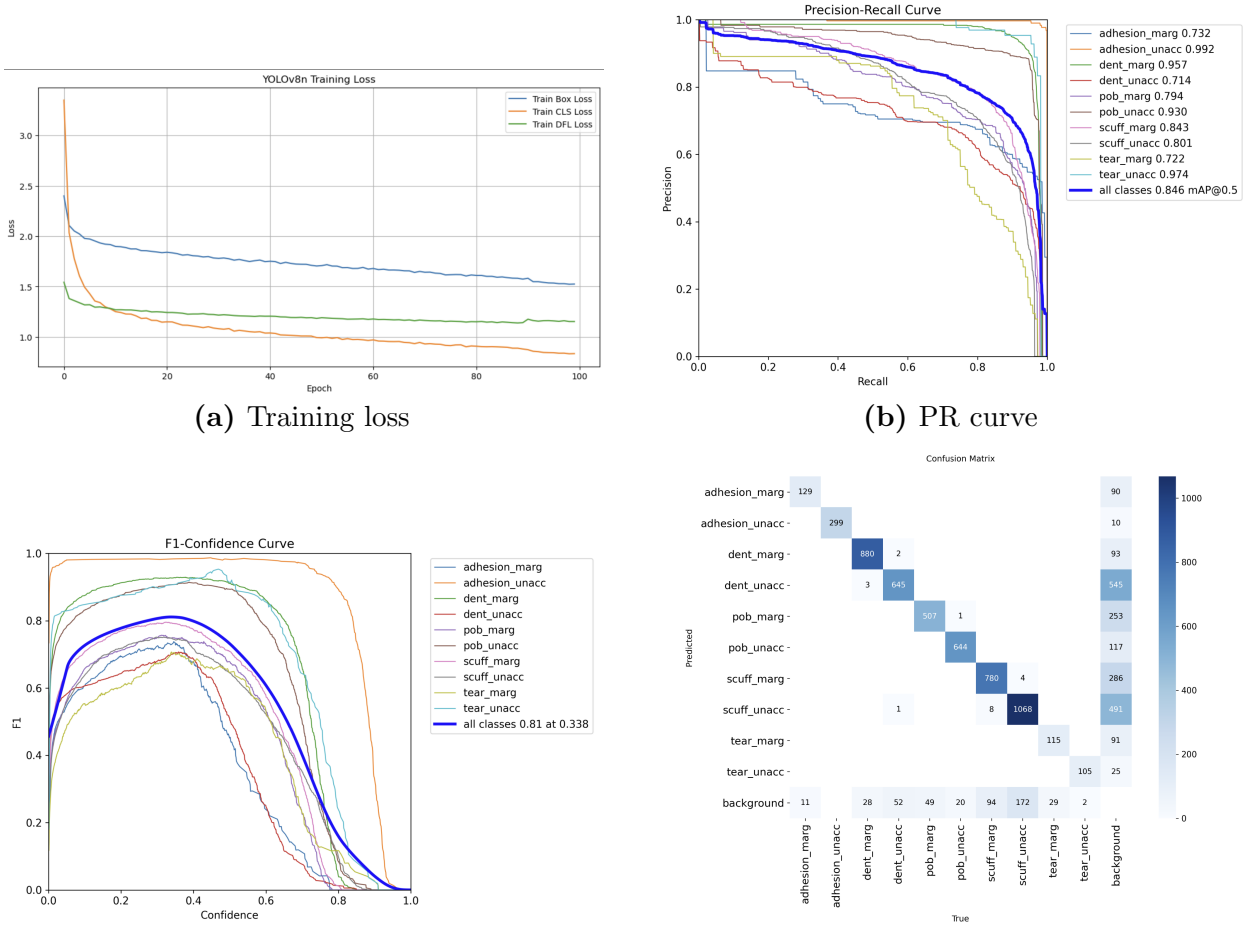
mAP50 is the mean Average Precision at IoU threshold 0.50. It is a common object detection metric. A high mAP50 means that the model detects and localizes objects well.

5.3 Training and Validation Plots

The performance of each model was evaluated using training and validation graphs. Loss graph helps understand if the model is learning through training process whereas validation graph and confusion matrix help understand the performance of the trained model on test data. These graphs have been organized per model such that each section contains only the plots of the corresponding model.

5.3.1 YOLOv8n Plots

YOLOv8n was used as a lightweight baseline model. Its plots include the training loss, precision-recall curve, F1 curve and confusion matrix. These plots help to understand both training behaviour and class-wise detection performance.



(a) Training loss **(b)** PR curve

(c) F1 curve **(d)** Confusion matrix

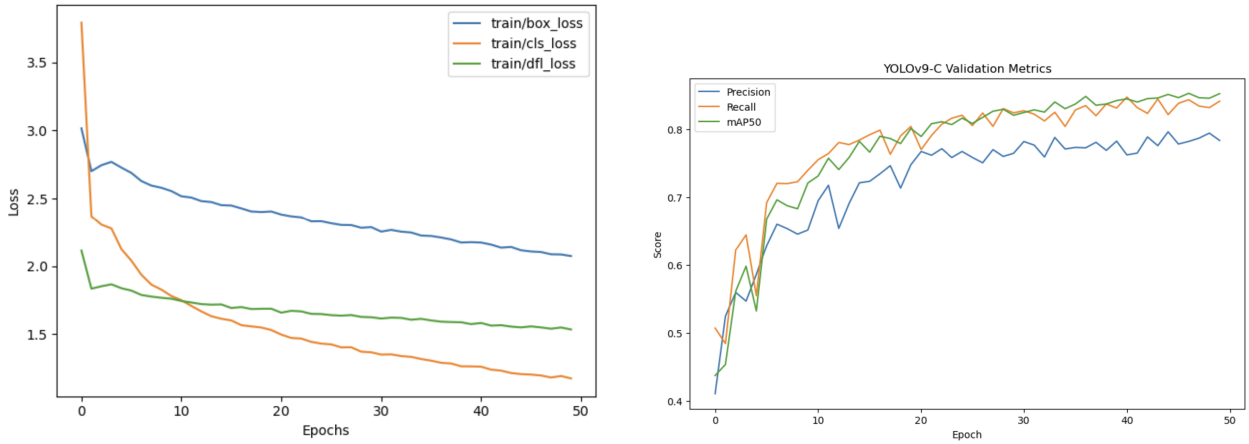
Figure 5.1: YOLOv8n training and validation plots.

The YOLOv8n plots show that the model learned useful defect features and achieved strong validation behaviour, especially in recall.

5.3.2 YOLOv9-C Plots

YOLOv9-C is the largest network in the comparison. The plot contains the training loss function and validation metrics for this network. These results indicate how well the larger network was able to learn during training and how its metrics evolved during validation. In particular, YOLOv9-C has many more layers and parameters than the nano networks, so it is better at learning complex patterns from the dataset of bottle defects. This is beneficial in case of defects like dents, scuffs, and tears, when the difference between marginal and unacceptable classes is small in terms of visual appearance. However, the larger architecture also means that the time required to train the network is longer. That is why the plot for YOLOv9-C is especially important.

YOLOv9-C achieved strong detection accuracy, but this came with much higher computational cost because of its larger architecture.

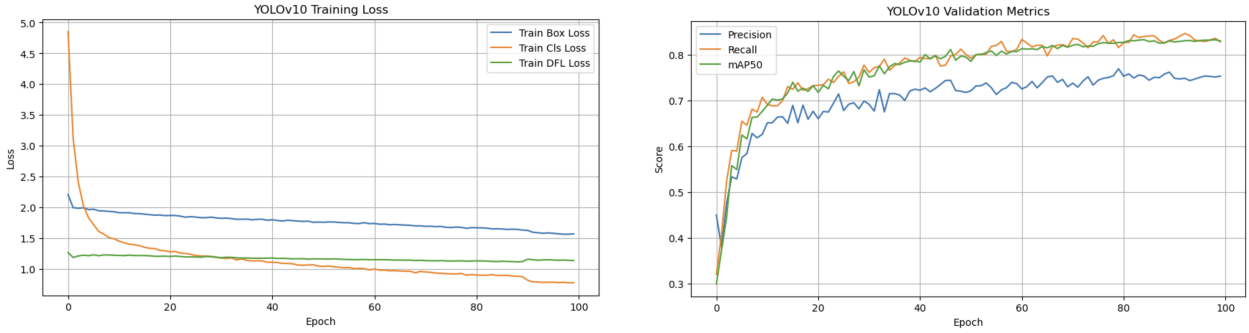


(a) Training loss **(b)** Validation metrics

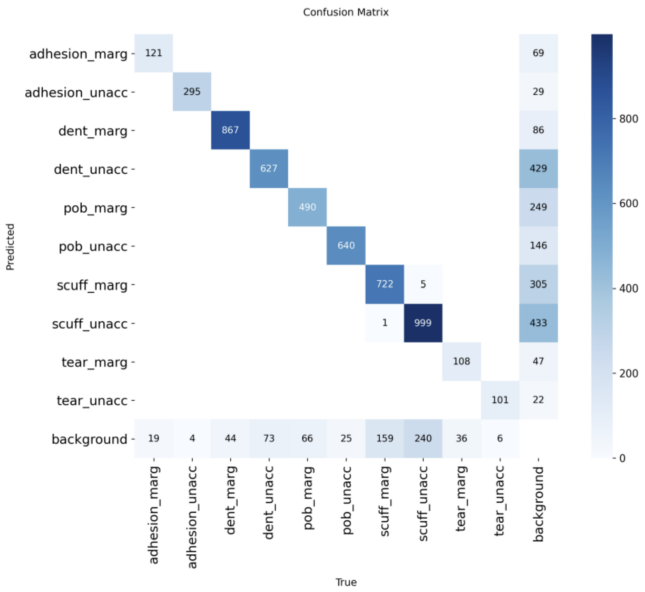
Figure 5.2: YOLOv9-C training loss and validation metrics.

5.3.3 YOLOv10n Plots

YOLOv10n was validated as an efficient end-to-end detector model. The plots for YOLOv10n include training loss and validation performance metrics and confusion matrices. This shows both learning process and final class-based detection performance.



(a) Training loss **(b)** Validation metrics



(c) Confusion matrix

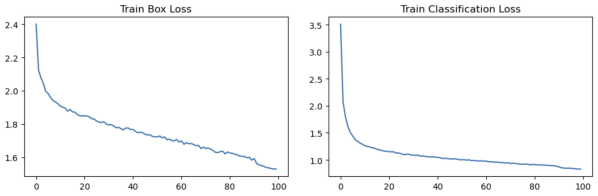
Figure 5.3: YOLOv10n training, validation and confusion matrix results.

YOLOv10n produced competitive accuracy while keeping fast inference behaviour. The

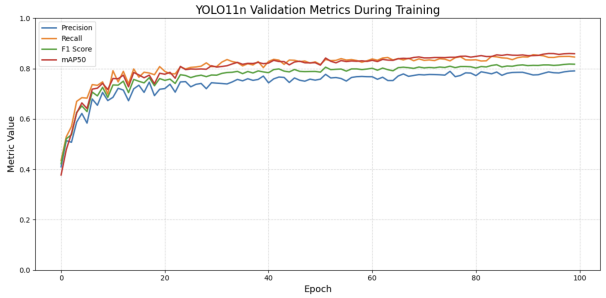
confusion matrix gives a class-level view of the detection results.

5.3.4 YOLO11n Plots

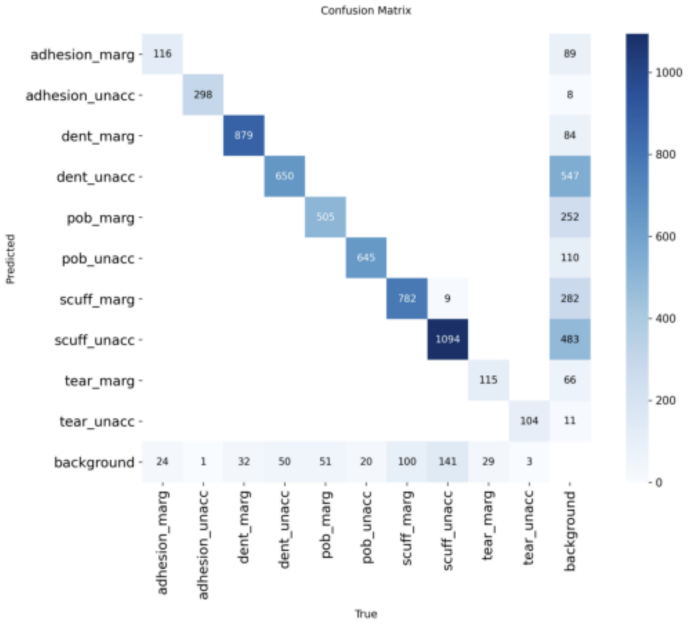
The most optimal model from this project is YOLO11n. The graphs for this model contain the loss for training and validation, as well as the confusion matrix. This makes the choice of YOLO11n very justified.



(a) Training loss components



(b) Validation metrics



(c) Confusion matrix

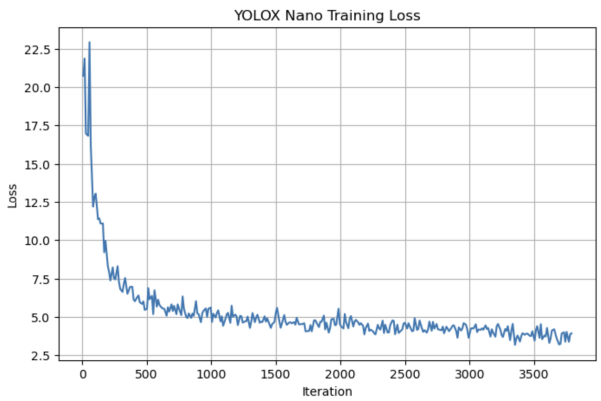
Figure 5.4: YOLO11n training, validation and confusion matrix results.

The YOLO11n plots show the strongest overall balance between precision, recall, F1-score and mAP50. The confusion matrix supports its selection as the best final model.

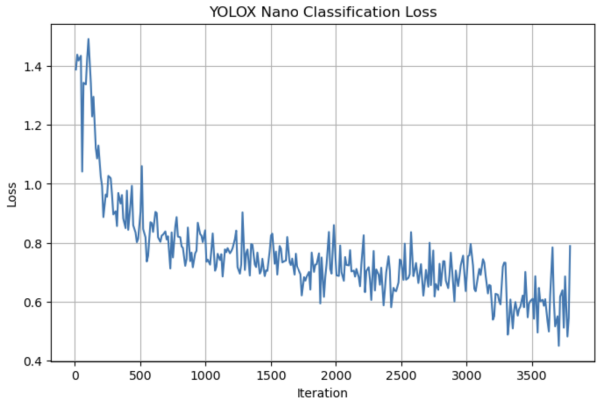
5.3.5 YOLOX-Nano Plots

The model named YOLOX-Nano was trained using YOLOX training framework. The plots that have been made by using the model include training loss, classification loss, and validation mAP50.

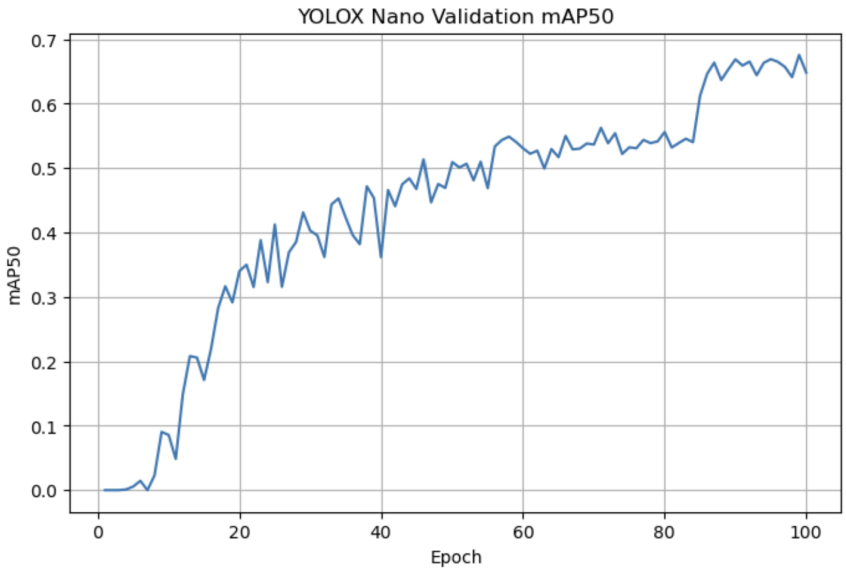
Losses on YOLOX-Nano decreased while the validation mAP50 increased with epoch progression, indicating that YOLOX-Nano did learn relevant detection features, albeit not as precise and accurate as the newer YOLO architectures.



(a) Training loss



(b) Classification loss



(c) Validation mAP50

Figure 5.5: YOLOX-Nano training loss, classification loss and validation mAP50.

Chapter 6

Results and Evaluation

6.1 Overall Test Results

Table 6.1: Final model comparison.

Model	Layers	Parameters	GFLOPs	Precision	Recall	F1	mAP50	Training Time
YOLOv8n	130	3.01M	8.2	0.779	0.857	0.815	0.846	63.22 min
YOLOv9-C	962	51.02M	239.0	0.783	0.841	0.811	0.852	320.40 min
YOLOv10n	224	2.71M	8.4	0.754	0.829	0.789	0.831	82.50 min
YOLO11n	182	2.59M	6.5	0.791	0.846	0.817	0.859	66.18 min
YOLOX-Nano	–	2.24M	6.94	0.474	0.474	0.474	0.797	100 epochs

6.2 Best Model for Each Metric

Table 6.2: Best model for each important metric.

Metric	Best Model	Explanation
Precision	YOLO11n	Highest precision.
Recall	YOLOv8n	Highest recall and fewer missed detections.
F1-score	YOLO11n	Best balance between precision and recall.
mAP50	YOLO11n	Highest detection accuracy at IoU 0.50.
Lowest GFLOPs	YOLO11n	Lowest computational cost
Smallest Parameters	YOLOX-Nano	Smallest model size.
Shortest Training Time	YOLOv8n	Fastest completed training time.

6.3 Detection Performance and Efficiency Ranking

Table 6.3: Ranking according to detection performance.

Rank	Model	F1	mAP50	Explanation
1	YOLO11n	0.817	0.859	Best overall detection result.
2	YOLOv8n	0.815	0.846	Very strong recall and short training time.
3	YOLOv9-C	0.811	0.852	High accuracy but very expensive computationally.
4	YOLOv10n	0.789	0.831	Fast and competitive, but lower F1.
5	YOLOX-Nano	0.474	0.797	Lightweight but weaker precision and recall.

Table 6.3 provides the ranking of the models with respect to the performance in terms of detection. YOLO11n model performed the best among all since it had the highest F1-score and mAP50 values. YOLOv8n model was close to it and had very good recall that is vital for detecting defects.

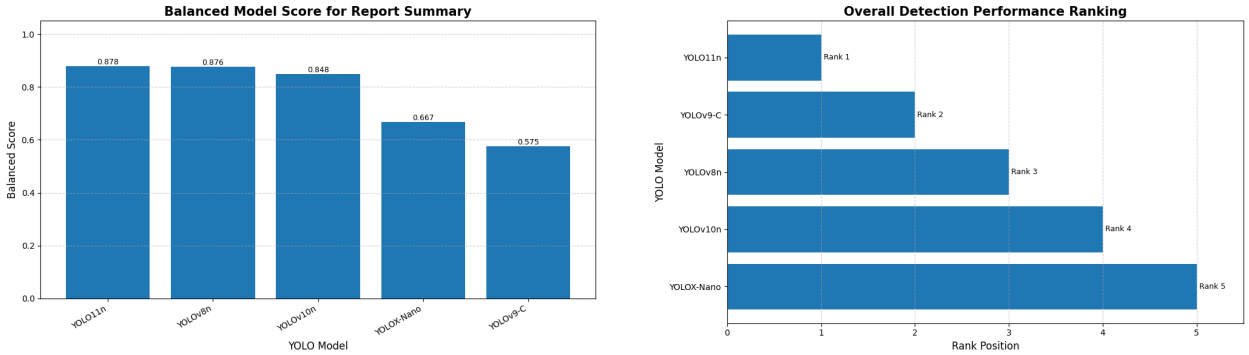
Table 6.4: Ranking according to computational efficiency.

Rank	Model	Params	GFLOPs	Explanation
1	YOLO11n	2.59M	6.5	Best accuracy-efficiency balance.
2	YOLOX-Nano	2.24M	6.94	Smallest model, but lower accuracy.
3	YOLOv8n	3.01M	8.2	Stable and accurate with moderate cost.
4	YOLOv10n	2.71M	8.4	Very fast inference with competitive accuracy.
5	YOLOv9-C	51.02M	239.0	Highest computational cost.

Table 6.4 provides ranking of models based on model size and computational efficiency. YOLO11n proves to be the most efficient practical solution due to its high accuracy and minimal GFLOPs usage among other efficient models. YOLOv9-C had decent detection accuracy, yet the huge number of parameters and GFLOPS reduces its efficiency.

6.4 Final Summary Graphs

This part provides a graphical representation of the final comparison of models. The figures provide a comparison of the models on the basis of the balanced score and overall detection ranking, along with the key detection measures. Figures will help in understanding the final outcome better.



(a) Balanced model score **(b)** Overall detection ranking

Figure 6.1: Final score and ranking summary for the tested YOLO models.

It is seen from Fig. 6.1 that YOLO11n was the winner with the best balanced score and YOLOv8n follows it with a minor gap. It is also worth mentioning that YOLOv10n had a good performance as compared to YOLOX-Nano and YOLOv9-C, which have less balanced scores due to low accuracy or high computational cost.

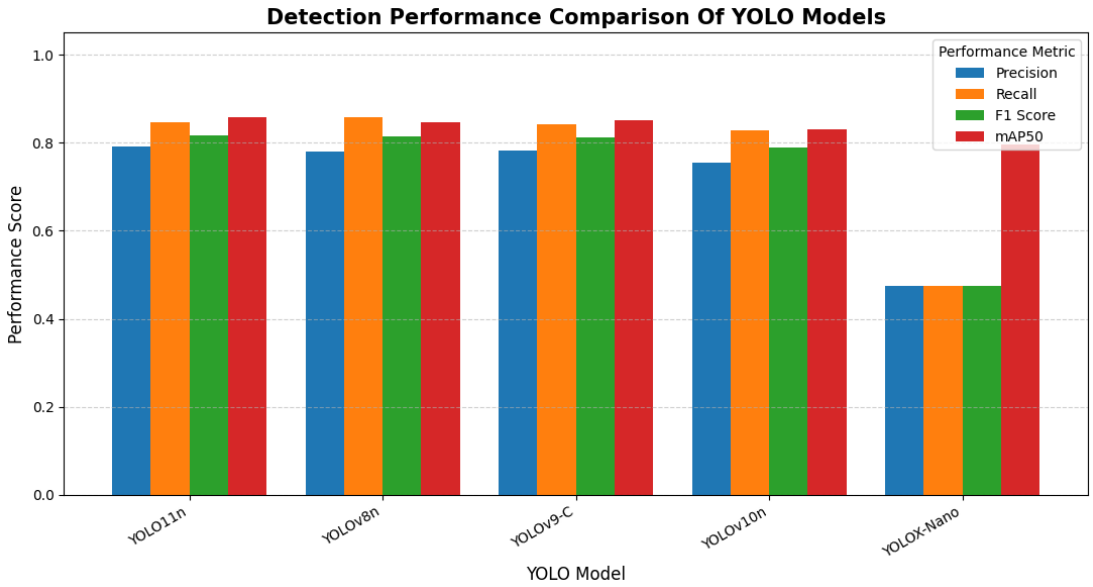


Figure 6.2: Detection performance comparison of YOLO models using precision, recall, F1-score and mAP50.

The Figure 6.2 below shows a comparative evaluation of the different detection metrics for all models tested. YOLO11n performed the best in all respects, having the highest F1-score and mAP50. The recall rate of YOLOv8n was very high, implying that the model performed well in detecting most defects. YOLOv9-C also offered good detection accuracy but the huge size of the model limits its effectiveness.

In general, the comparative graphs above back the conclusion made that YOLO11n is the best model for this work.

Chapter 7

Discussion, Conclusion and Future Work

7.1 Discussion

YOLO11n obtained the most favorable detection results. It had the largest F1-score and mAP50 values while maintaining a minimum number of parameters and GFLOPs. In other words, YOLO11n not only demonstrated high accuracy but also showed efficiency.

YOLOv8n proved to be the most effective model in terms of the combination of accuracy and learning speed. This model had the largest recall value that is very important in defect detection since missing the object is more harmful than false detection.

YOLOv9-C provided high accuracy results, although it used significantly more computations. The larger model size enabled learning good features; however, it took much more computations compared to any other model. It reduces its applicability due to its large computational costs.

Inference speed of YOLOv10n was extremely fast, also the accuracy of the results was rather competitive. The design is aimed at reducing the post-processing procedures to increase efficiency. Nevertheless, in this case, the results of F1-score and mAP50 are slightly worse compared to YOLOv8n and YOLO11n.

Though YOLOX-Nano was the lightest weight model and required the lowest computation cost, it had relatively poor precision and recall results. This proves that a very small model lacks the ability to handle the challenging defect classes with similar appearances.

The most significant outcome here is that the selection of the model cannot be done considering only its accuracy. An ideal model for an industry must not only have high efficiency and stability, but also be easily deployable. Here, YOLO11n is the best model.

7.2 Conclusion

The analysis focused on comparing five YOLO models for plastic shampoo bottle defect detection. All five models were trained and tested using the same 10 class defects dataset. The comparison parameters covered accuracy measurements, dimensions of the models, computation cost, optimization technique, activation function, and fine-tuning capabilities.

The end results revealed that YOLO11n was the best performer. YOLOv8n also performed well and trained fast. YOLOv9-C was highly accurate but was costly to compute. YOLOv10n was highly efficient and suited for efficient usage scenarios. YOLOX-Nano was compact but not highly accurate.

7.3 Future Work

Further steps for research include increasing the number of images in defect categories that have fewer samples. It is also worthwhile to explore class imbalance solutions. Further experiments with data augmentation, hyperparameters, and explainable AI (Grad-CAM) techniques can also help understand which areas of the image are important for making predictions.

Appendix: Sample Detection Outputs

This appendix shows sample detection outputs from the trained YOLO models. The images show predicted bounding boxes and confidence scores on plastic shampoo bottle defect samples. They are included to visually support the quantitative results discussed in the evaluation chapter.

YOLO11n Detection Output

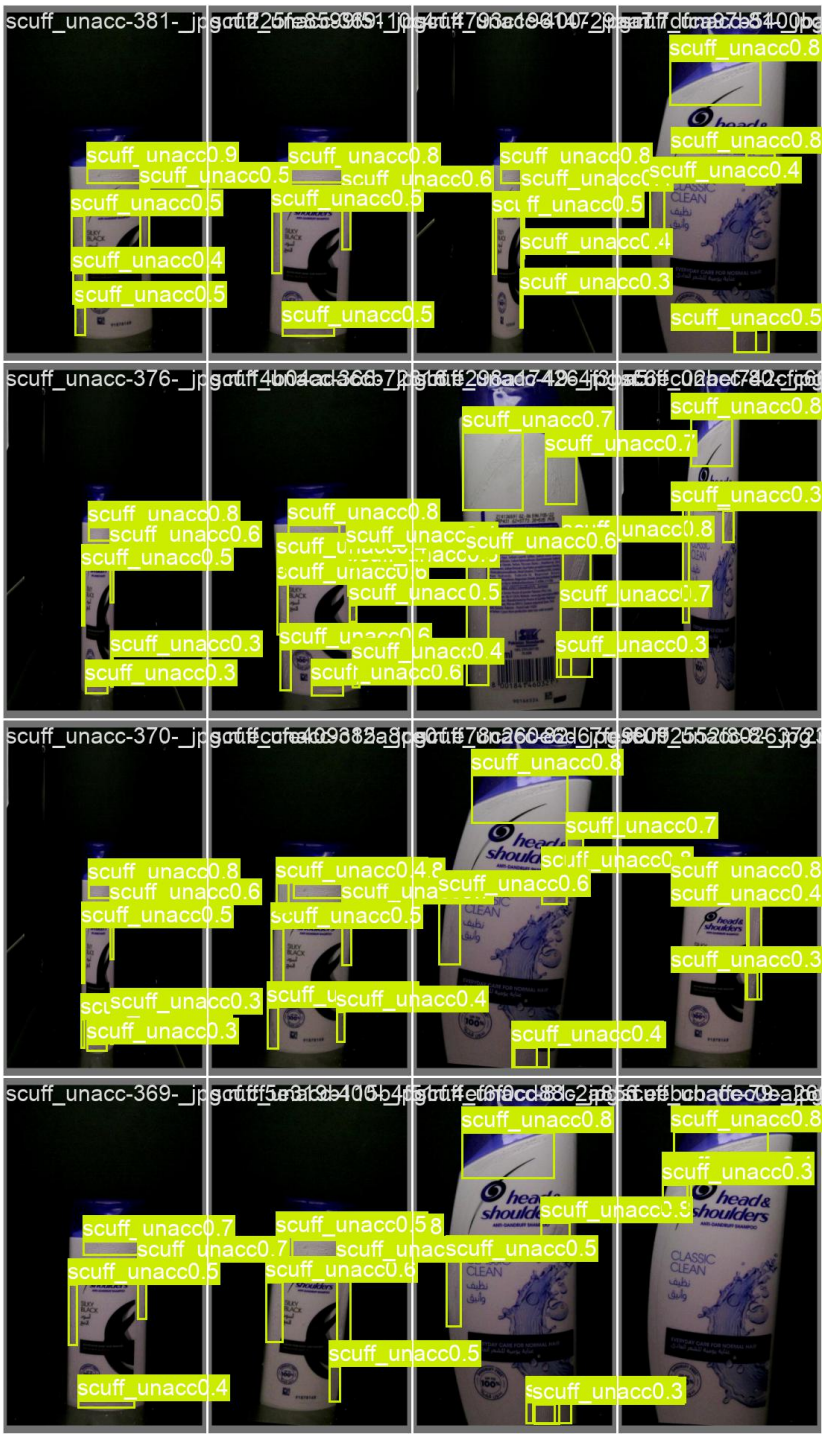


Figure 7.1: Sample detection output from YOLO11n.

YOLOv10n Detection Output

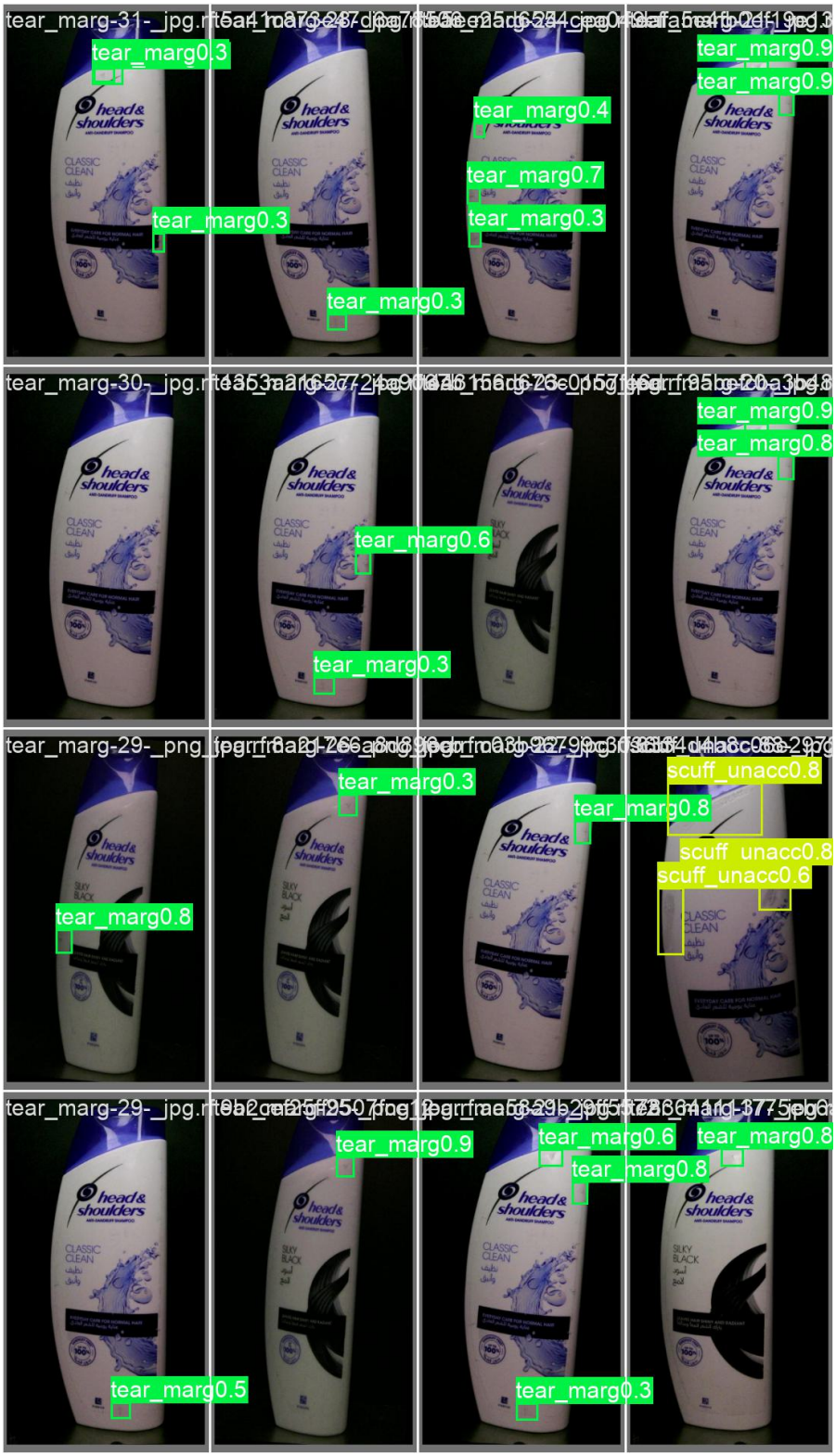


Figure 7.2: Sample detection output from YOLOv10n.

YOLOv8n Detection Output

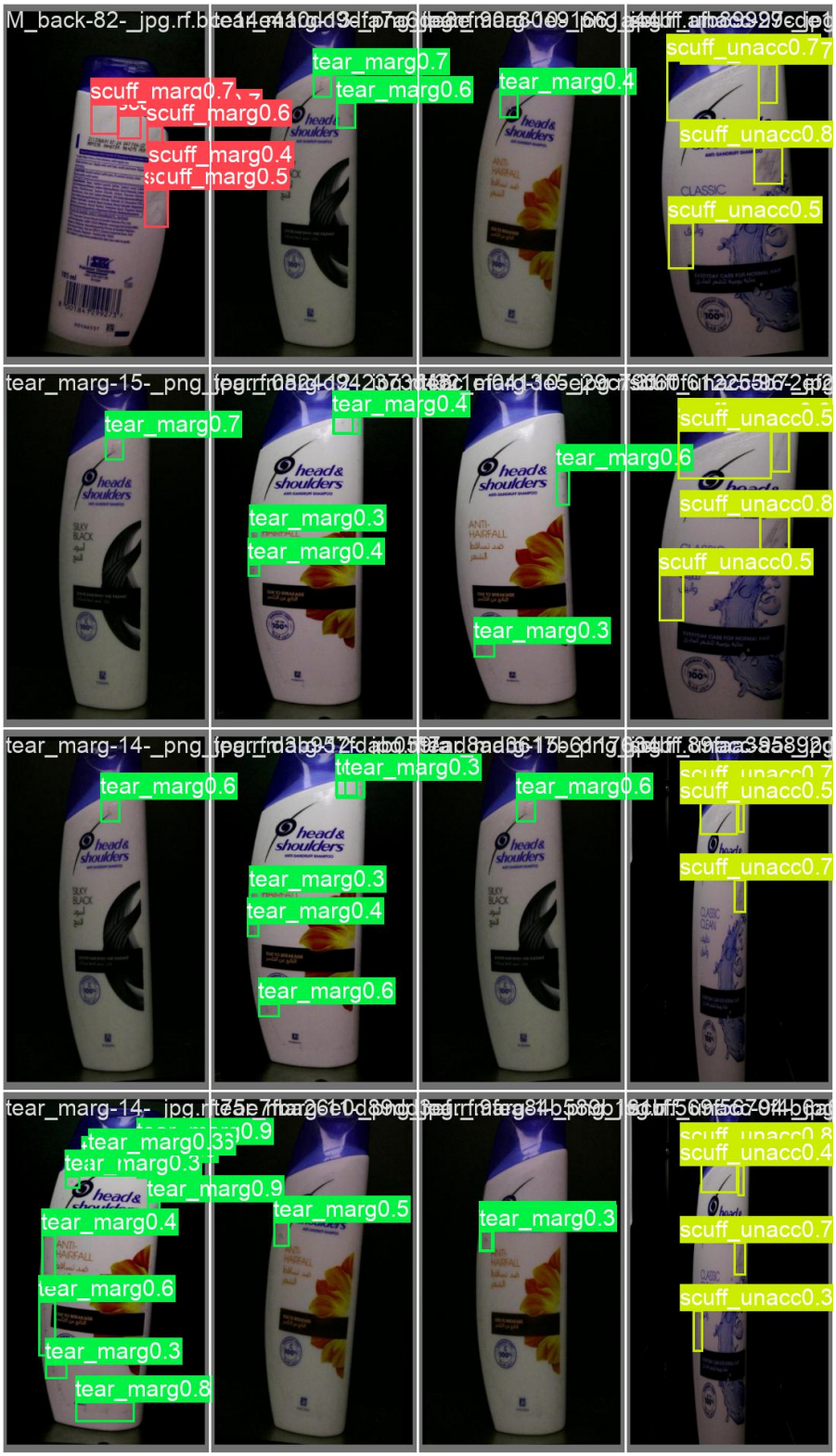


Figure 7.3: Sample detection output from YOLOv8n.

Bibliography

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[2] Ultralytics, "Ultralytics YOLOv8 Documentation," 2024. [Online]. Available: `https://docs.ultralytics.com/models/yolov8/`

[3] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information," *European Conference on Computer Vision (ECCV)*, 2024. [Online]. Available: `https://arxiv.org/abs/2402.13616`

[4] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-Time End-to-End Object Detection," arXiv preprint arXiv:2405.14458, 2024. [Online]. Available: `https://arxiv.org/abs/2405.14458`

[5] Ultralytics, "Ultralytics YOLO11 Documentation," 2024. [Online]. Available: `https://docs.ultralytics.com/models/yolo11/`

[6] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO Series in 2021," arXiv preprint arXiv:2107.08430, 2021. [Online]. Available: `https://arxiv.org/abs/2107.08430`

[7] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How Transferable are Features in Deep Neural Networks?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[8] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[9] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, "Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[10] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression," in *AAAI Conference on Artificial Intelligence*, 2020.